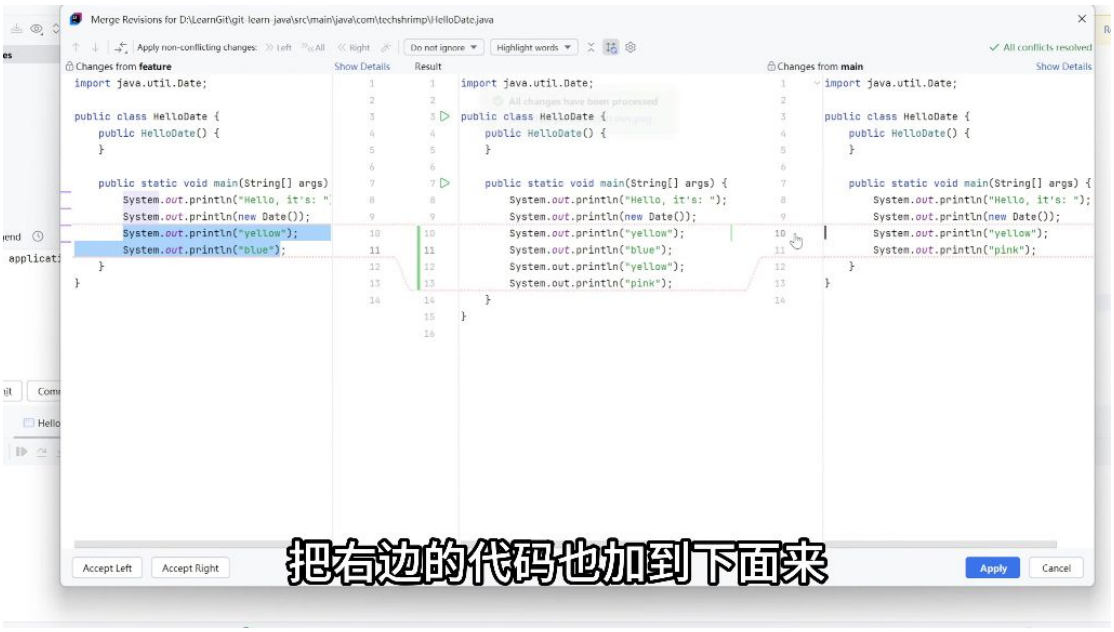
-- 以下为AI生成的图文笔记内容 --

*一、针对**Git**的进阶操作**00:00***

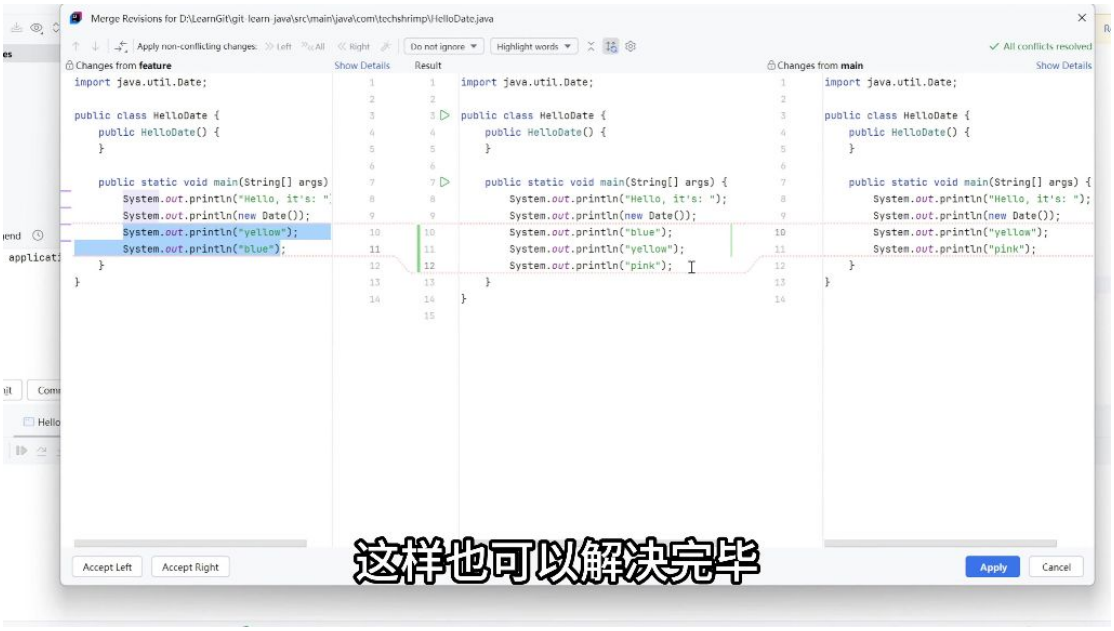
1. 解决冲突 00:05

-
- **冲突处理方式**：
  - Accept Yours：保留当前分支的修改，覆盖对方分支的修改
  - Accept Theirs：保留对方分支的修改，覆盖当前分支的修改
  - Merge：手动合并双方的修改
- **IDE优势**：
  - 相比命令行需要手动修改冲突文件，IDE提供了可视化界面
  - 可以直观看到双方分支的修改内容
  - 通过点击按钮即可选择保留哪方的修改

- 

- **详细操作**：
  - 左侧窗口显示当前分支的修改
  - 右侧窗口显示对方分支的修改
  - 中间窗口是最终合并结果
  - 可以分别选择接受左侧、右侧或同时接受两边的修改

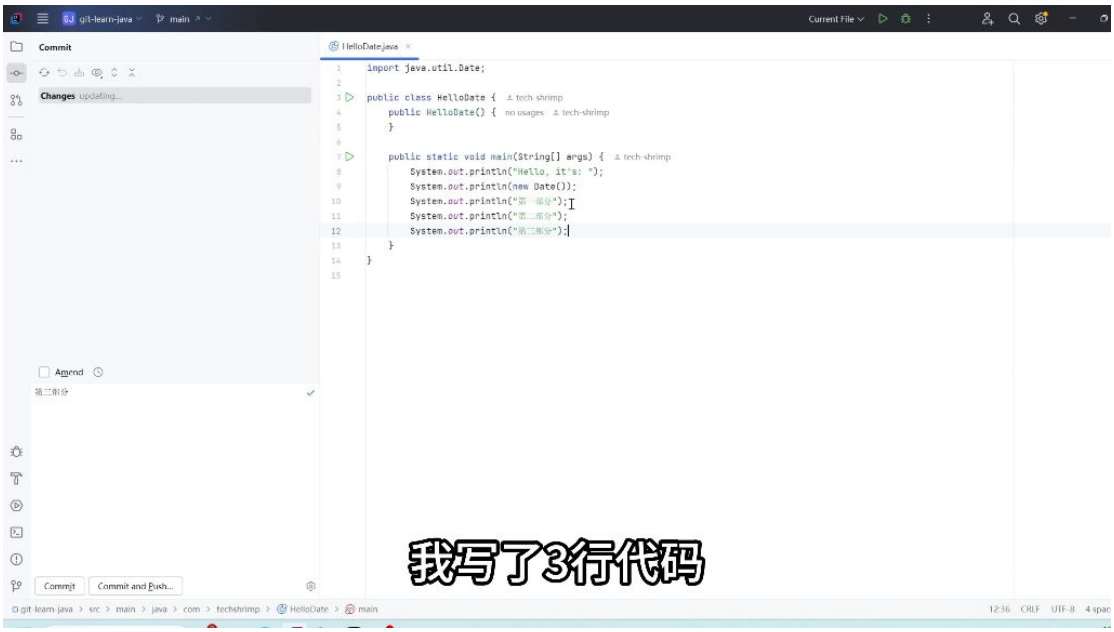
2. 合并分支 02:42

- 

- **合并步骤**：
  - 在分支列表中选择要合并的分支

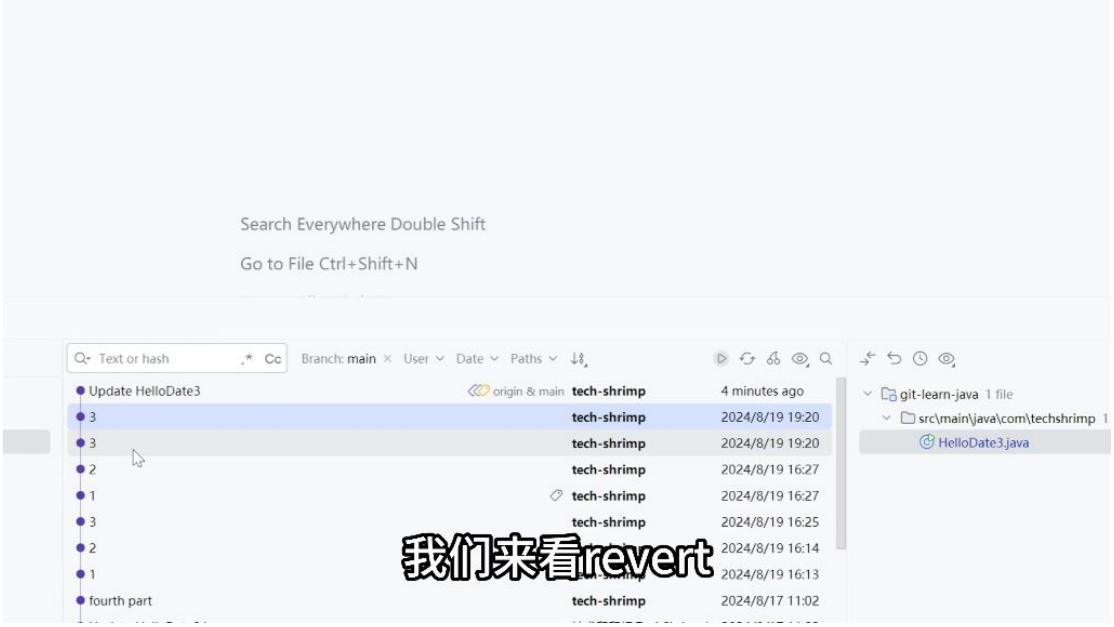
    - 右键点击选择"Merge into current"
    - 处理可能出现的冲突
    - 提交合并结果
- **注意事项**：
    - 合并前确保工作区是干净的
    - 合并后需要将结果推送到远程仓库

3. 回退修改 04:49

- 

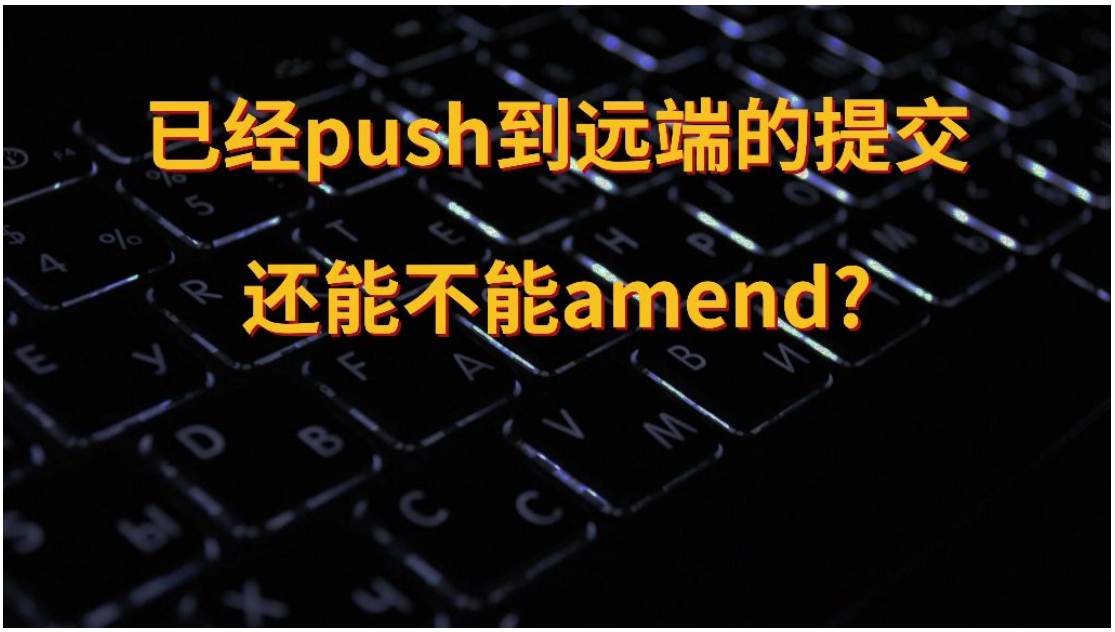
- **Rollback功能**：
    - 等同于git的discard操作
    - 会丢弃本地文件的修改
    - 使本地文件与本地分支强制同步
- **使用场景**：
    - 当修改出错需要完全放弃时
    - 想恢复到上次提交的状态时

4. 撤销提交 07:16

- 

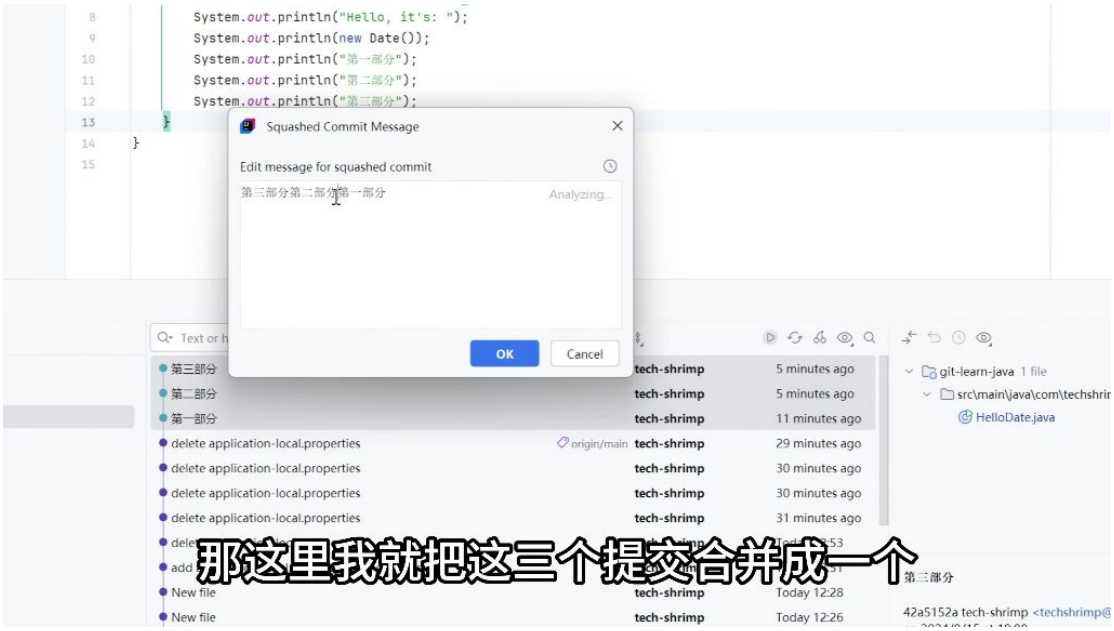
- **reset类型**：
  - soft：仅撤销提交，修改保留在暂存区和工作目录
  - mixed（默认）：撤销提交和暂存区，修改保留在工作目录
  - hard：完全撤销提交、暂存区和工作目录的修改
  - keep：撤销提交和暂存区，但保留未提交的修改
- **推荐方式**：
  - 通常使用mixed模式，这是最安全的方式
  - hard会丢失所有未提交的修改，慎用

5. 修改最近的提交 08:05

- 

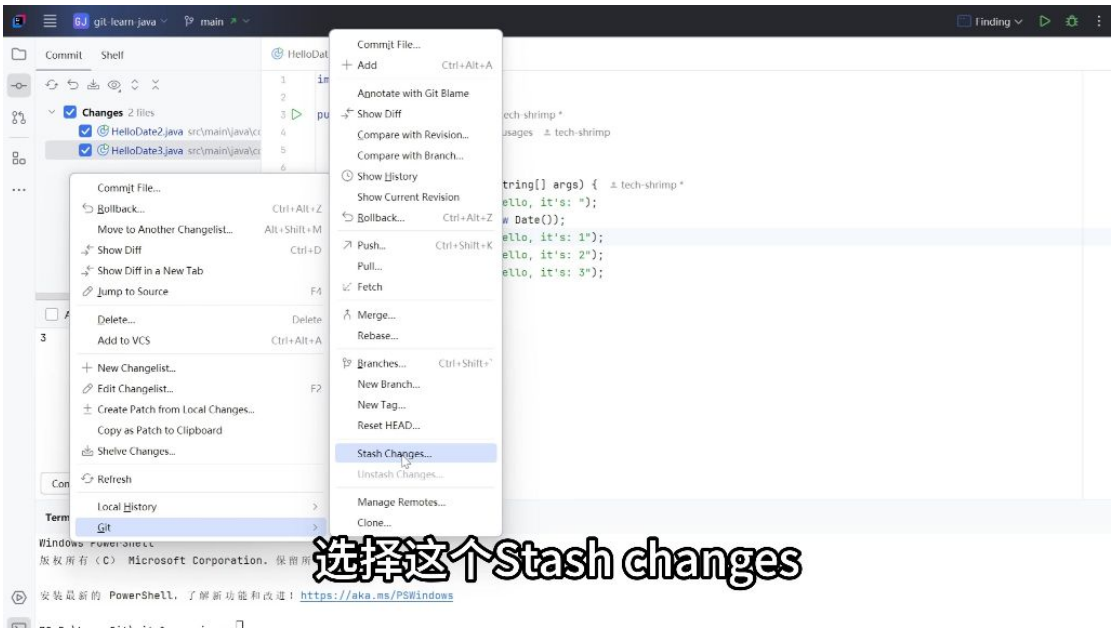
- **amend操作**：
  - 只能修改最近的一次提交
  - 可以修改提交信息和文件内容
  - 不会产生新的提交记录
- **限制**：
  - 已经推送到远端的提交不能直接amend
  - 如需修改远端提交需要强制推送，但不推荐

6. 合并提交 09:34

- 

- **squash功能**：
  - 将多个提交合并为一个
  - 可以整理提交历史，使其更清晰
  - 需要为合并后的提交编写新的提交信息
- **注意事项**：
  - 只能合并尚未推送到远端的提交
  - 已推送到远端的提交不能直接squash

7. 存储更改 10:46

- 

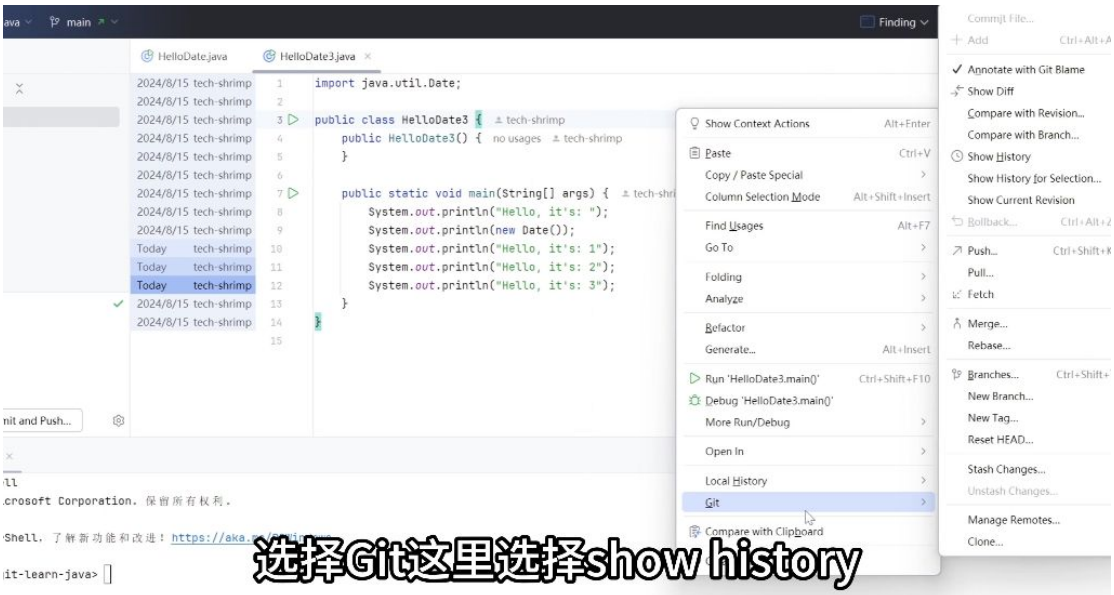
- **stash功能**：
  - 临时保存工作区的修改
  - 允许切换分支而不提交当前修改
  - 可以添加描述信息便于识别
- **局限性**：
  - 一次性存储所有文件的更改
  - 不能选择性存储部分文件

8. 搁置更改 11:39

-

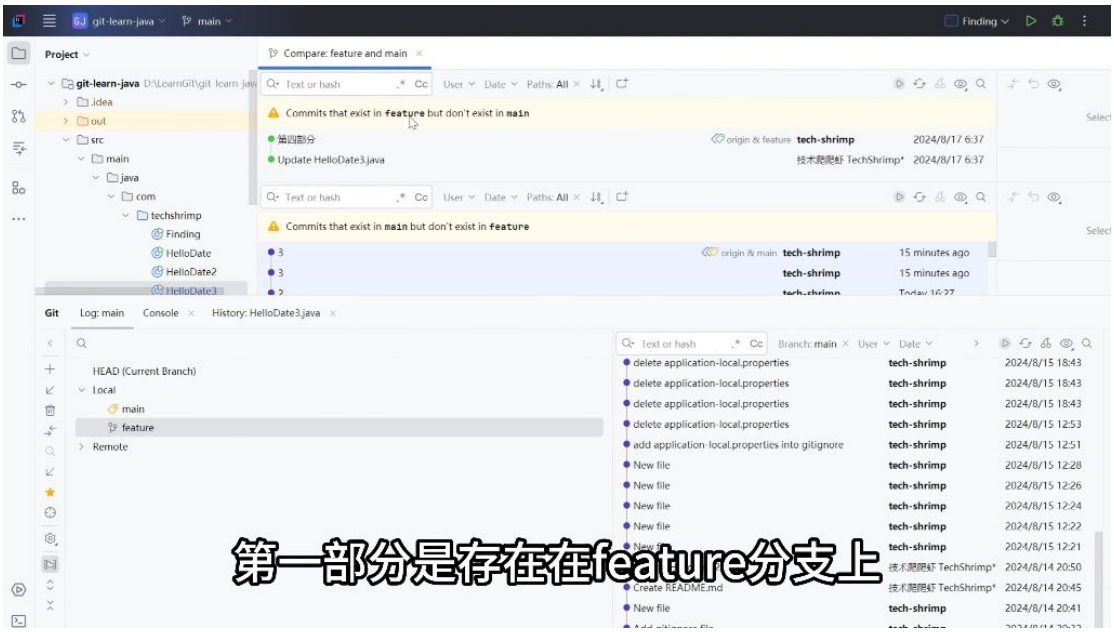
- **shelve功能**：
  - IDE特有的功能，类似stash但更灵活
  - 可以选择性搁置部分文件的更改
  - 搁置的更改可以随时恢复
- **优势**：
  - 比stash更精细的控制
  - 适合需要临时保存部分修改的场景

9. git blame 12:26



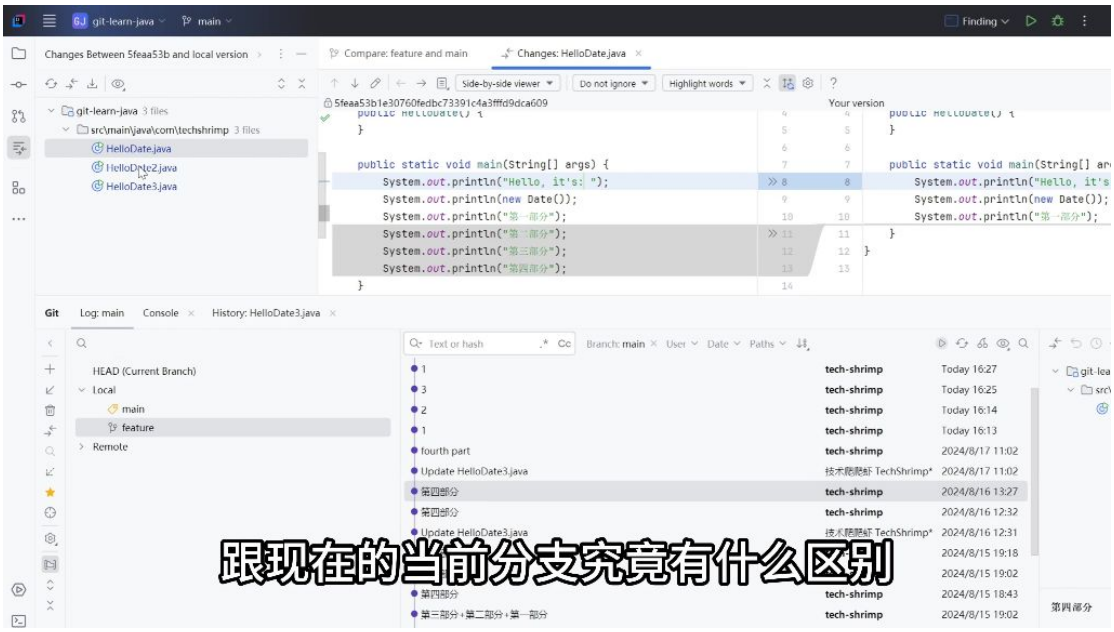
-
- **功能**：
  - 显示文件中每行代码的最后修改者和修改时间
  - 便于追踪代码变更历史
  - 可以通过GUI或命令行使用
- **命令行用法**：
  - git blame <文件名>：查看整个文件的修改历史
  - git blame -L <开始行>,<结束行> <文件名>：查看指定行的修改历史

10. 比较分支 13:53

- 

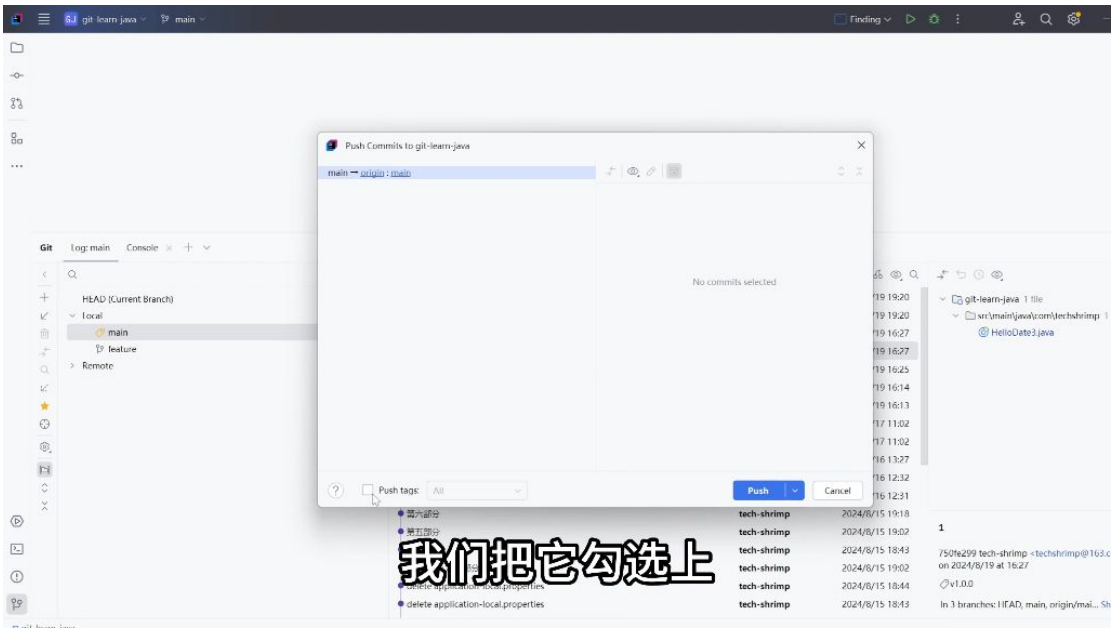
- **比较内容**：
  - 显示存在于一个分支但不在另一个分支的提交
  - 可以查看每个提交的详细差异
  - 帮助理解分支间的差异
- **操作步骤**：
  - 右键点击要比较的分支
  - 选择"Compare with current"
  - 查看差异分析结果

11. 比较提交 14:18

- 

- **功能**：
  - 比较任意两次提交之间的差异
  - 可以查看文件级别的变更
  - 支持并排显示差异
- **使用场景**：
  - 检查特定版本间的变化
  - 定位引入问题的提交
  - 了解代码演进过程

12. 创建标签 14:43

- 

- **标签类型**：
  - 轻量标签：只是一个指向特定提交的引用
  - 附注标签：包含创建者、日期和描述信息
- **推送标签**：
  - 默认情况下git push不会推送标签
  - 需要显式推送标签到远程仓库
  - 可以推送单个标签或所有标签

*二、结束**15:19***

三、知识小结

知识点	核心内容	关键操作/技巧	注意事项
冲突解决	IDEA可视化解决Git冲突	Accept Yours/Theirs选择 多窗口代码比对合并	可选择性保留双方代码块 需手动确认最终合并结果
分支更新	两种更新策略对比	Merge（保留合并记录） Rebase（线性历史）	**Rebase更适合团队协作** 避免合并提交污染历史
代码回退	多维度撤销更改	Roll Back（文件级） Reset（提交级） Revert（提交反转）	Hard reset会丢失工作区修改 Revert生成新提交记录

提交修改	修正提交记录	Amend（修改最近提交） Squash（合并多个提交）	**已推送提交禁止amend** Squash需在本地未推送时使用
代码暂存	灵活存储修改	Stash（全量暂存） Shelve（选择性暂存）	Shelve可指定文件暂存 Stash适用于全量上下文保存
历史追溯	代码变更追踪	Blame（行级追溯） Show History（文件历史）	支持命令行/GUI双模式 可限定追溯行号范围
差异比对	多维度差异分析	分支对比 提交版本对比	可视化展示变更文件 支持具体内容差异比对
版本标记	里程碑管理	Tag创建与推送	遵循语义化版本规范 需显式推送Tags到远程